
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 7

Ponteiros em C

Exercícios (7.7–7.31)

Resolvidos passo a passo, com código completo

Contents

1	Exercício 7.7 – Preenchimento de Lacunas	2
2	Exercício 7.8 – Verdadeiro ou Falso	2
3	Exercício 7.9 – Array values	2
4	Exercício 7.10 – long e ponteiros	4
5	Exercício 7.11 – Cabeçalhos e Protótipos	4
6	Exercício 7.12 – Embaralhamento + Pôquer (5 cartas)	4
7	Exercício 7.13 – Duas mãos de pôquer	7
8	Exercícios 7.14 e 7.15 – Carteador e Jogo Completo	9
9	Exercício 7.16 – Embaralhamento Eficiente	9
10	Exercício 7.17 – A Tartaruga e a Lebre	11
11	Exercício 7.18 – Embaralhar + Distribuir em uma função	13
12	Exercício 7.19 – O que mystery1 faz?	13
13	Exercício 7.20 – O que mystery2 faz?	14
14	Exercício 7.21 – Erros	15
15	Exercício 7.22 – Travessia de Labirinto	17
16	Exercício 7.23 – Gerar Labirintos	18
17	Exercício 7.24 – Labirintos de qualquer tamanho	18
18	Exercício 7.25 – Arrays de Ponteiros para Funções	19
19	Exercício 7.26 – O que mystery3 faz?	22
20	Exercícios 7.27 a 7.29 – Simpletron	22
21	Exercício 7.30 – Cálculos Geométricos com Ponteiros para Função	25
22	Exercício 7.31 – Cálculos com Dois Operandos	26

1. Exercício 7.7 – Preenchimento de Lacunas

Resposta detalhada

- a) O operador `&` (endereço, ou *address-of*) retorna o local na memória onde o operando está armazenado.
- b) O operador `*` (desreferência, ou *indireção*) retorna o valor do objeto apontado pelo operando.
- c) Para simular chamada por referência ao passar variável não-array, é necessário passar o(a) **endereço** da variável (usando `&variavel`).

2. Exercício 7.8 – Verdadeiro ou Falso

Resposta detalhada

- a) **Verdadeiro.** Comparar ponteiros que apontam para arrays diferentes *não tem significado prático*. As comparações só fazem sentido para ponteiros dentro do mesmo array, onde pode-se verificar qual elemento vem antes/depois.
- b) **Falso.** Embora o nome do array atue como ponteiro para o primeiro elemento, ele é um **ponteiro constante** – não pode ser modificado. Tentar `++arr` ou `arr = outroPonteiro` gera erro de compilação. Já uma variável-ponteiro pode ser livremente modificada.

3. Exercício 7.9 – Array values

Resposta detalhada

Considere: `unsigned int = 2 bytes`; endereço inicial = 1002500; `#define SIZE 5`.

a)

```
1 unsigned int values[ SIZE ] = { 2, 4, 6, 8, 10 };
```

b)

```
1 unsigned int *vPtr;
```

c)

```
1 for ( i = 0; i < SIZE; ++i )
2     printf( "%u ", values[ i ] );
```

d)

```
1 vPtr = values;
2 vPtr = &values[ 0 ];
```

e)

```
1 for ( i = 0; i < SIZE; ++i )
2     printf( "%u ", *( vPtr + i ) );
```

f)

```
1 for ( i = 0; i < SIZE; ++i )
2     printf( "%u ", *( values + i ) );
```

g)

```
1 for ( i = 0; i < SIZE; ++i )
2     printf( "%u ", vPtr[ i ] );
```

h) Elemento 5 (último, índice 4):

```
1 values[ 4 ]           /* notacao de subscrito de array
2     */
3 *( values + 4 )       /* nome do array como ponteiro
4     */
5 vPtr[ 4 ]             /* notacao de subscrito de
6     ponteiro */
7 *( vPtr + 4 )         /* ponteiro/deslocamento
8     */
```

i) Endereço de `vPtr + 3`: $1002500 + 3 \times 2 = \mathbf{1002506}$; valor: `values[3]` = 8.

j) Endereço após `vPtr -= 4` (`vPtr` aponta para `values[4]`):

- Endereço de `values[4]`: $1002500 + 4 \times 2 = 1002508$
- Após `vPtr -= 4`: $1002508 - 4 \times 2 = \mathbf{1002500}$

Esse é o endereço de `values[0]`, valor: **2**.

4. Exercício 7.10 – long e ponteiros

Resposta detalhada

```
a) long *lPtr;
b) lPtr = &value1;
c) printf( "%ld\n", *lPtr );
d) value2 = *lPtr;
e) printf( "%ld\n", value2 );
f) printf( "%p\n", &value1 );
g) printf( "%p\n", lPtr ); – Sim, é o mesmo endereço de value1, pois
    lPtr foi inicializado em (b).
```

5. Exercício 7.11 – Cabeçalhos e Protótipos

Resposta detalhada

a) Cabeçalho de zero:

```
1 void zero( long bigIntegers[] )
```

b) Protótipo de zero:

```
1 void zero( long bigIntegers[] );
```

c) Cabeçalho de add1AndSum:

```
1 int add1AndSum( int oneTooSmall[] )
```

d) Protótipo:

```
1 int add1AndSum( int oneTooSmall[] );
```

Nota: em C, tipo nome[] em parâmetro de função é equivalente a tipo *nome. Ambos significam “ponteiro para o tipo”.

6. Exercício 7.12 – Embaralhamento + Pôquer (5 cartas)

Resposta detalhada

Estratégia: estender o programa de embaralhar/distribuir cartas para distribuir mãos de 5 cartas e detectar combinações de pôquer.

```
1  /* Ex7.12: poker.c -- esqueleto com funcoes principais
   * /
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <time.h>
5  #define CARTAS 52
6  #define MAO 5
7
```

```
8  /* Tipos de combinacoes */
9  int temPar          ( const int mao[] );
10 int temDoisPares    ( const int mao[] );
11 int temTrinca       ( const int mao[] );
12 int temQuadra       ( const int mao[] );
13 int temFlush        ( const int mao[] );
14 int temStraight     ( const int mao[] );
15
16 /* As cartas vem representadas por inteiros 0..51:
17    valor = carta % 13    (0=A, 1=2, ..., 12=K)
18    naipe = carta / 13    (0=Ouros, 1=Espadas, 2=Copas,
19                          3=Paus) */
19
20 /* Conta frequencias de cada valor (0..12) na mao */
21 void contaValores( const int mao[], int freq[] )
22 {
23     int i;
24     for ( i = 0; i < 13; ++i ) freq[ i ] = 0;
25     for ( i = 0; i < MAO; ++i ) {
26         ++freq[ mao[ i ] % 13 ];
27     }
28 }
29
30 int temPar( const int mao[] )
31 {
32     int freq[ 13 ], i;
33     contaValores( mao, freq );
34     for ( i = 0; i < 13; ++i ) {
35         if ( freq[ i ] == 2 ) return 1;
36     }
37     return 0;
38 }
39
40 int temDoisPares( const int mao[] )
41 {
42     int freq[ 13 ], i, pares = 0;
43     contaValores( mao, freq );
44     for ( i = 0; i < 13; ++i ) {
45         if ( freq[ i ] == 2 ) ++pares;
46     }
47     return pares == 2;
48 }
49
50 int temTrinca( const int mao[] )
51 {
52     int freq[ 13 ], i;
53     contaValores( mao, freq );
54     for ( i = 0; i < 13; ++i ) {
55         if ( freq[ i ] == 3 ) return 1;
56     }
57 }
```

```
57     return 0;
58 }
59
60 int temQuadra( const int mao[] )
61 {
62     int freq[ 13 ], i;
63     contaValores( mao, freq );
64     for ( i = 0; i < 13; ++i ) {
65         if ( freq[ i ] == 4 ) return 1;
66     }
67     return 0;
68 }
69
70 int temFlush( const int mao[] )
71 {
72     int naipe = mao[ 0 ] / 13;
73     int i;
74     for ( i = 1; i < MAO; ++i ) {
75         if ( mao[ i ] / 13 != naipe ) return 0;
76     }
77     return 1;
78 }
79
80 int temStraight( const int mao[] )
81 {
82     int valores[ MAO ];
83     int i, j, temp;
84
85     /* Extrai e ordena os valores */
86     for ( i = 0; i < MAO; ++i ) valores[ i ] = mao[ i
87         ] % 13;
88     for ( i = 0; i < MAO - 1; ++i ) {
89         for ( j = 0; j < MAO - 1 - i; ++j ) {
90             if ( valores[ j ] > valores[ j + 1 ] ) {
91                 temp = valores[ j ];
92                 valores[ j ] = valores[ j + 1 ];
93                 valores[ j + 1 ] = temp;
94             }
95         }
96     }
97
98     /* Verifica se sao consecutivos */
99     for ( i = 0; i < MAO - 1; ++i ) {
100         if ( valores[ i + 1 ] != valores[ i ] + 1 )
101             return 0;
102     }
103     return 1;
104 }
```

7. Exercício 7.13 – Duas mãos de pôquer

Resposta detalhada

Estratégia: distribuir 10 cartas (2 mãos de 5), avaliar cada mão e determinar a melhor.

```
1  /* Ex7.13: comparar_maos.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <time.h>
5  #define CARTAS 52
6  #define MAO 5
7
8  /* Inclui funcoes de Ex7.12: temPar, temFlush, ... */
9
10 /* Retorna pontuacao da mao (maior = melhor) */
11 int avaliaMao( const int mao[] )
12 {
13     if ( temFlush( mao ) && temStraight( mao ) )
14         return 8; /* Straight Flush */
15     if ( temQuadra( mao ) )
16         return 7; /* Quadra */
17     if ( temTrinca( mao ) && temPar( mao ) )
18         return 6; /* Full House */
19     if ( temFlush( mao ) )
20         return 5; /* Flush */
21     if ( temStraight( mao ) )
22         return 4; /* Sequencia */
23     if ( temTrinca( mao ) )
24         return 3; /* Trinca */
25     if ( temDoisPares( mao ) )
26         return 2; /* Dois pares */
27     if ( temPar( mao ) )
28         return 1; /* Par */
29     return 0; /* Carta alta */
30 }
31
32 const char *nomeMao( int valor )
33 {
34     switch ( valor ) {
35         case 8: return "Straight Flush";
36         case 7: return "Quadra";
37         case 6: return "Full House";
38         case 5: return "Flush";
39         case 4: return "Sequencia";
40         case 3: return "Trinca";
41         case 2: return "Dois pares";
42         case 1: return "Par";
43         default: return "Carta alta";
44     }
45 }
```

```
37 }
38
39 int main( void )
40 {
41     int baralho[ CARTAS ];
42     int mao1[ MAO ], mao2[ MAO ];
43     int i, temp, idx;
44     int v1, v2;
45
46     srand( time( NULL ) );
47
48     /* Inicializa e embaralha (Fisher-Yates) */
49     for ( i = 0; i < CARTAS; ++i ) baralho[ i ] = i;
50     for ( i = CARTAS - 1; i > 0; --i ) {
51         idx = rand() % ( i + 1 );
52         temp = baralho[ i ];
53         baralho[ i ] = baralho[ idx ];
54         baralho[ idx ] = temp;
55     }
56
57     /* Distribui 5 para cada mao */
58     for ( i = 0; i < MAO; ++i ) {
59         mao1[ i ] = baralho[ i ];
60         mao2[ i ] = baralho[ i + MAO ];
61     }
62
63     v1 = avaliaMao( mao1 );
64     v2 = avaliaMao( mao2 );
65
66     printf( "Mao 1: %s\n", nomeMao( v1 ) );
67     printf( "Mao 2: %s\n", nomeMao( v2 ) );
68
69     if ( v1 > v2 ) printf( "Mao 1 vence!\n" );
70     else if ( v2 > v1 ) printf( "Mao 2 vence!\n" );
71     else printf( "Empate na categoria.\n" );
72
73     return 0;
74 }
```

8. Exercícios 7.14 e 7.15 – Carteador e Jogo Completo

Resposta detalhada

7.14 – Simular o carteador trocando cartas:

Após avaliar a mão do carteador, decida quantas cartas trocar:

- Se tem quadra ou melhor: *não troca* (mão muito boa).
- Se tem trinca: troca 2 cartas restantes.
- Se tem dois pares: troca 1 carta (a do meio).
- Se tem um par: troca 3 cartas.
- Sem par algum: troca 3 cartas (mantém as 2 mais altas).

7.15 – Jogo completo (jogador vs. computador):

```

1  /* Esqueleto */
2  1. Embaralhe.
3  2. Distribua 5 cartas para o jogador e 5 para o
   carteador.
4  3. Mostre as cartas do jogador; carteador fica oculto.
5  4. Pergunte ao jogador quais cartas trocar (índices).
6  5. Substitua cartas do jogador.
7  6. O carteador escolhe automaticamente (regras do
   7.14)
8     quais trocar e troca.
9  7. Compare mãos e exiba vencedor.

```

Reflexão: estes exercícios mostram como ponteiros e arrays compõem sistemas complexos. O jogo de pôquer envolve combinatória, estratégia (do carteador) e interatividade. Refinamentos sucessivos permitem ajustar a heurística do computador.

9. Exercício 7.16 – Embaralhamento Eficiente

Resposta detalhada

Problema do algoritmo da Figura 7.24: ele escolhe linha/coluna aleatórios e *tenta* colocar a carta – se a posição já está ocupada, tenta outra, podendo iterar indefinidamente (**adiamento indefinido**).

Solução eficiente (*Fisher-Yates Shuffle*):

```

1  /* Ex7.16: embaralhar eficiente -- O(n) */
2  void embaralhaEficiente( int baralho[][ 13 ] )
3  {
4      int i, j, ni, nj, temp;
5      /* Inicializa: posicao (linha, coluna) contem a
6         carta original */
7
8      /* Percorre todas as 52 posicoes */
9      for ( i = 0; i < 4; ++i ) {
10         for ( j = 0; j < 13; ++j ) {
11             /* Escolhe posicao aleatoria QUALQUER do

```

```
11         baralho */
12         ni = rand() % 4;
13         nj = rand() % 13;
14         /* Troca */
15         temp = baralho[ i ][ j ];
16         baralho[ i ][ j ] = baralho[ ni ][ nj ];
17         baralho[ ni ][ nj ] = temp;
18     }
19 }
```

Vantagem: cada elemento é tocado exatamente uma vez ($O(n)$). Não há risco de adiamento indefinido. Esta é a essência do algoritmo de Fisher-Yates, usado em todos os geradores profissionais de embaralhamento.

Distribuição eficiente: em vez de procurar a carta 1, depois a 2, etc., basta percorrer o array sequencialmente. Após embaralhar, as cartas *já estão* em ordem aleatória – a posição no array *é* a ordem de distribuição.

10. Exercício 7.17 – A Tartaruga e a Lebre

Resposta detalhada

```
1  /* Ex7.17: tartaruga_lebre.c
2     Simulacao classica da corrida com geracao aleatoria
3     */
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <time.h>
7  #define PISTA 70
8
9  void moveTartaruga( int *posicao );
10 void moveLebre     ( int *posicao );
11 void exhibeCorrida ( int tartaruga, int lebre );
12
13 int main( void )
14 {
15     int tartaruga = 1, lebre = 1;
16     int ticks = 0;
17
18     srand( time( NULL ) );
19
20     printf( "BANG !!!!!\nE LA VAO ELES !!!!!\n" );
21
22     while ( tartaruga < PISTA && lebre < PISTA ) {
23         moveTartaruga( &tartaruga );
24         moveLebre( &lebre );
25         exhibeCorrida( tartaruga, lebre );
26         ++ticks;
27     }
28
29     printf( "\n" );
30     if ( tartaruga >= PISTA && lebre >= PISTA ) {
31         printf( "E um empate.\n" );
32     }
33     else if ( tartaruga >= PISTA ) {
34         printf( "TARTARUGA VENCE!!! E ISSO AI!!!\n" );
35     }
36     else {
37         printf( "Lebre vence. Marmelada.\n" );
38     }
39     printf( "Total de batidas: %d\n", ticks );
40
41     return 0;
42 }
43
44 void moveTartaruga( int *posicao )
45 {
46     int i = 1 + rand() % 10;
```

```

46     if      ( i >= 1 && i <= 5 ) *posicao += 3;    /*
        caminha rapido */
47     else if ( i == 6 || i == 7 ) *posicao -= 6;    /*
        escorrega      */
48     else                                     *posicao += 1;    /*
        caminha devagar*/
49
50     if ( *posicao < 1 ) *posicao = 1;
51 }
52
53 void moveLebre( int *posicao )
54 {
55     int i = 1 + rand() % 10;
56     if      ( i >= 1 && i <= 2 ) ;                  /*
        dorme: nao move */
57     else if ( i >= 3 && i <= 4 ) *posicao += 9;      /*
        grande salto   */
58     else if ( i == 5 )          *posicao -= 12;    /*
        escorregao     */
59     else if ( i >= 6 && i <= 8 ) *posicao += 1;      /*
        pulinho        */
60     else                          *posicao -= 2;    /*
        escorrega pouco */
61
62     if ( *posicao < 1 ) *posicao = 1;
63 }
64
65 void exhibeCorrida( int tartaruga, int lebre )
66 {
67     int i;
68     for ( i = 1; i <= PISTA; ++i ) {
69         if ( i == tartaruga && i == lebre ) printf( "
            AI" );
70         else if ( i == tartaruga ) printf( "T" );
71         else if ( i == lebre ) printf( "L" );
72         else printf( " " );
73     }
74     printf( "\n" );
75 }

```

Conceito-chave do Cap. 7: usamos `*posicao` dentro de `moveTartaruga` e `moveLebre` para alterar a variável original. Essa é a **chamada por referência simulada** – exatamente o caso de uso central de ponteiros!

11. Exercício 7.18 – Embaralhar + Distribuir em uma função

Resposta detalhada

Combina-se a estrutura aninhada da `shuffle` com a lógica da `deal` em uma única função que percorre cada carta uma vez, embaralhando enquanto distribui. Essa abordagem é mais elegante e evita o adiamento indefinido. A ideia central: ao percorrer o array sequencialmente após embaralhar (Fisher-Yates), a posição é a ordem natural de distribuição. As primeiras 5 são a mão 1, as próximas 5 são a mão 2, e assim por diante.

12. Exercício 7.19 – O que `mystery1` faz?

Resposta detalhada

```
1 void mystery1( char *s1, const char *s2 )
2 {
3     while ( *s1 != '\0' ) { /* primeira fase: avançar
4                             ate o fim de s1 */
5         s1++;
6     }
7     for ( ; *s1 = *s2; s1++, s2++ ) {
8         ; /* corpo vazio */
9     }
```

Análise:

- a) Primeira fase: o ponteiro `s1` avança até atingir o `'\0'` (terminador da string).
- b) Segunda fase: copia caractere por caractere de `s2` para o local apontado por `s1`, incluindo o `'\0'` de `s2`.

Conclusão: `mystery1` é uma implementação manual de `strcat` – concatena `s2` ao final de `s1`.

Exemplo: se `string1 = "Ola"` e `string2 = "Mundo"`, ao final `string1 = "OlaMundo"`.

Saída final: `OlaMundo`.

Nota: a expressão `*s1 = *s2` na condição do `for` é uma atribuição (não comparação!) que avalia para o caractere copiado. Quando o `'\0'` é copiado, a condição se torna falsa e o loop termina.

13. Exercício 7.20 – O que mystery2 faz?

Resposta detalhada

```
1 int mystery2( const char *s )
2 {
3     int x;
4     for ( x = 0; *s != '\0'; s++ ) {
5         x++;
6     }
7     return x;
8 }
```

Análise: A função percorre cada caractere de `s` até encontrar `'\0'`, incrementando `x` a cada caractere.

Conclusão: `mystery2` é uma implementação manual de `strlen` – retorna o *comprimento da string* (número de caracteres antes do `'\0'`).

Exemplo: `mystery2("Hello")` retorna 5.

14. Exercício 7.21 – Erros

Resposta detalhada

a) Código:

```
1 int *number;  
2 printf( "%d\n", *number );
```

Erro: ponteiro não inicializado, desreferenciado. **Correção:** inicializar antes:

```
1 int x = 42;  
2 int *number = &x;  
3 printf( "%d\n", *number );
```

b) Código:

```
1 float *realPtr;  
2 long *integerPtr;  
3 integerPtr = realPtr;
```

Erro: atribuição entre ponteiros de tipos incompatíveis. **Correção:** usar cast (mas verificar se faz sentido):

```
1 integerPtr = ( long * )realPtr;
```

c) Código:

```
1 int * x, y;  
2 x = y;
```

Erro: `int * x, y;` declara `x` como `int *` e `y` como `int` simples. Atribuir um `int` (`y`) a um ponteiro (`x`) é inválido. **Correção:**

```
1 int *x, *y; /* ambos sao ponteiros */  
2 x = y;      /* OK */
```

Ou, se a intenção era atribuir endereço de `y` a `x`:

```
1 int *x;  
2 int y;  
3 x = &y;
```

d) Código:

```
1 char s[] = "este e um array de caracteres";  
2 int count;  
3 for ( ; *s != '\0'; s++ )  
4     printf( "%c ", *s );
```

Erro: `s` é nome de array – não pode ser incrementado. **Correção:** usar ponteiro auxiliar:

```
1 char *p = s;  
2 for ( ; *p != '\0'; p++ )  
3     printf( "%c ", *p );
```

e) Código:

```
1 short *numPtr, result;  
2 void *genericPtr = numPtr;  
3 result = *genericPtr + 7;
```

Erro: desreferência de void * sem cast. **Correção:**

```
1 result = *( ( short * )genericPtr ) + 7;
```

f) Código:

```
1 float x = 19.34;  
2 float xPtr = &x;  
3 printf( "%f\n", xPtr );
```

Erros: (1) xPtr declarado como float (deveria ser float *); (2) imprime ponteiro com %f. **Correção:**

```
1 float *xPtr = &x;  
2 printf( "%f\n", *xPtr ); /* ou usar %p para o  
   endereco */
```

g) Código:

```
1 char *s;  
2 printf( "%s\n", s );
```

Erro: s é ponteiro não inicializado. **Correção:**

```
1 char *s = "Hello, World!";  
2 printf( "%s\n", s );
```

15. Exercício 7.22 – Travessia de Labirinto

Resposta detalhada

Algoritmo da mão direita: sempre seguir a parede da direita garante a saída de qualquer labirinto simplesmente conexo. Implementação recursiva:

```

1  /* Ex7.22: mazeTraverse.c -- esqueleto */
2  #include <stdio.h>
3  #define LADO 12
4
5  /* Direcoes: 0=Norte, 1=Leste, 2=Sul, 3=Oeste */
6  int dRow[] = { -1, 0, 1, 0 };
7  int dCol[] = { 0, 1, 0, -1 };
8
9  int mazeTraverse( char m[][ LADO ], int row, int col,
10 int dir );
11 void exhibeLab( char m[][ LADO ] );
12
13 int mazeTraverse( char m[][ LADO ], int row, int col,
14 int dir )
15 {
16     m[ row ][ col ] = 'X';
17     exhibeLab( m );
18
19     /* Tenta primeiro a direita, depois a frente,
20 depois esquerda */
21     int tentativas[] = {
22         ( dir + 1 ) % 4,    /* direita */
23         dir,                /* frente */
24         ( dir + 3 ) % 4,    /* esquerda */
25         ( dir + 2 ) % 4     /* tras */
26     };
27
28     int t, novaR, novaC, novaD;
29     for ( t = 0; t < 4; ++t ) {
30         novaD = tentativas[ t ];
31         novaR = row + dRow[ novaD ];
32         novaC = col + dCol[ novaD ];
33
34         if ( novaR < 0 || novaR >= LADO ||
35             novaC < 0 || novaC >= LADO ) {
36             /* Saiu do labirinto = encontrou a saída!
37             */
38             return 1;
39         }
40
41         if ( m[ novaR ][ novaC ] == '.' ) {
42             if ( mazeTraverse( m, novaR, novaC, novaD ) )
43                 return 1;
44         }
45     }
46 }

```

```
40     }
41 }
42
43 /* Beco sem saída: desmarca e retrocede */
44 m[ row ][ col ] = '.';
45 return 0;
46 }
```

16. Exercício 7.23 – Gerar Labirintos

Resposta detalhada

A geração aleatória de labirintos é um problema complexo. Algoritmos clássicos:

- **DFS recursivo:** começar com tudo parede, escolher célula inicial, explorar vizinhos não-visitados aleatoriamente, removendo paredes.
- **Algoritmo de Prim:** similar, mas escolhe sempre uma parede aleatória da fronteira.
- **Algoritmo de Kruskal:** usa *union-find*.

A função `mazeGenerator` deve fornecer também as coordenadas de entrada e saída do labirinto, geralmente nos extremos do tabuleiro.

17. Exercício 7.24 – Labirintos de qualquer tamanho

Resposta detalhada

Para generalizar, basta passar o tamanho como parâmetro:

```
1 int mazeTraverse( char **m, int linhas, int colunas,
2                  int row, int col, int dir );
```

Em C, arrays bidimensionais de tamanho variável podem ser tratados como **ponteiros para ponteiros** (`char **`) ou alocados dinamicamente com `malloc` (Cap. 12).

18. Exercício 7.25 – Arrays de Ponteiros para Funções

Resposta detalhada

```
1  /* Ex7.25: menu_grades.c
2     Menu com array de ponteiros para funcoes */
3  #include <stdio.h>
4  #define ALUNOS 3
5  #define EXAMES 4
6
7  void printArray( const int grades[][ EXAMES ] );
8  void minimum   ( const int grades[][ EXAMES ] );
9  void maximum   ( const int grades[][ EXAMES ] );
10 void average    ( const int grades[][ EXAMES ] );
11
12 int main( void )
13 {
14     /* Array de ponteiros para funcoes */
15     void ( *processGrades[ 4 ] )( const int [][] EXAMES
16         ] ) = {
17         printArray, minimum, maximum, average
18     };
19
20     int grades[ ALUNOS ][ EXAMES ] = {
21         { 77, 68, 86, 73 },
22         { 96, 87, 89, 81 },
23         { 70, 90, 86, 81 }
24     };
25
26     int escolha;
27
28     do {
29         printf( "\nDigite uma escolha:\n" );
30         printf( " 0 Imprime o array de notas\n" );
31         printf( " 1 Acha a menor nota\n" );
32         printf( " 2 Acha a maior nota\n" );
33         printf( " 3 Imprime a media de cada aluno\n" );
34         ;
35         printf( " 4 Encerra o programa\n" );
36         printf( "Escolha: " );
37         scanf( "%d", &escolha );
38
39         if ( escolha >= 0 && escolha < 4 ) {
40             ( *processGrades[ escolha ] )( grades );
41         }
42     } while ( escolha != 4 );
43
44     return 0;
45 }
```

```
45 void printArray( const int grades[][ EXAMES ] )
46 {
47     int aluno, exame;
48     for ( aluno = 0; aluno < ALUNOS; ++aluno ) {
49         for ( exame = 0; exame < EXAMES; ++exame ) {
50             printf( "%4d ", grades[ aluno ][ exame ] )
51             ;
52         }
53         printf( "\n" );
54     }
55
56 void minimum( const int grades[][ EXAMES ] )
57 {
58     int aluno, exame, baixa = 100;
59     for ( aluno = 0; aluno < ALUNOS; ++aluno ) {
60         for ( exame = 0; exame < EXAMES; ++exame ) {
61             if ( grades[ aluno ][ exame ] < baixa )
62                 baixa = grades[ aluno ][ exame ];
63         }
64     }
65     printf( "Menor nota: %d\n", baixa );
66 }
67
68 void maximum( const int grades[][ EXAMES ] )
69 {
70     int aluno, exame, alta = 0;
71     for ( aluno = 0; aluno < ALUNOS; ++aluno ) {
72         for ( exame = 0; exame < EXAMES; ++exame ) {
73             if ( grades[ aluno ][ exame ] > alta )
74                 alta = grades[ aluno ][ exame ];
75         }
76     }
77     printf( "Maior nota: %d\n", alta );
78 }
79
80 void average( const int grades[][ EXAMES ] )
81 {
82     int aluno, exame, soma;
83     for ( aluno = 0; aluno < ALUNOS; ++aluno ) {
84         soma = 0;
85         for ( exame = 0; exame < EXAMES; ++exame ) {
86             soma += grades[ aluno ][ exame ];
87         }
88         printf( "Aluno %d: media = %.2f\n",
89                 aluno + 1, ( double )soma / EXAMES );
90     }
91 }
```

Chave do exercício: processGrades[i] contém um ponteiro para

uma das quatro funções. A escolha do usuário (0, 1, 2, 3) seleciona qual chamar – substituindo um `switch` extenso por uma única linha: `(*processGrades[escolha])(grades)`.

19. Exercício 7.26 – O que mystery3 faz?

Resposta detalhada

```
1 int mystery3( const char *s1, const char *s2 )
2 {
3     for ( ; *s1 != '\0' && *s2 != '\0'; s1++, s2++ ) {
4         if ( *s1 != *s2 ) {
5             return 0;
6         }
7     }
8     return 1;
9 }
```

Análise: A função compara caracteres correspondentes de `s1` e `s2`, terminando ao primeiro `'\0'` encontrado em qualquer das strings. Se algum par for diferente, retorna 0. Caso contrário, retorna 1.

Conclusão: `mystery3` verifica se duas strings são **iguais** (análoga à parte “de igualdade” de `strcmp`). **Atenção:** se uma string é prefixo da outra (ex.: “casa” e “casamento”), a função retorna 1 incorretamente, porque o loop termina assim que uma das strings acaba.

Versão corrigida:

```
1 return *s1 == *s2; /* verifica que ambos chegaram em
   \0 */
```

20. Exercícios 7.27 a 7.29 – Simpletron

Resposta detalhada

Visão geral: estes três exercícios formam um projeto integrado: construir um simulador completo de uma máquina virtual chamada **Simpletron**, com sua própria linguagem de máquina (SML).

Componentes do Simpletron:

- **Memória:** 100 palavras de 4 dígitos com sinal.
- **Acumulador:** registrador especial para cálculos.
- **Contador de instrução:** aponta para próxima instrução.
- **Registrador de instrução:** contém a instrução atual.

Conjunto de instruções SML:

Código	Nome	Função
10	READ	lê palavra do teclado
11	WRITE	imprime palavra
20	LOAD	carrega palavra no acumulador
21	STORE	armazena acumulador na memória
30	ADD	soma à palavra do acumulador
31	SUBTRACT	subtrai do acumulador
32	DIVIDE	divide o acumulador
33	MULTIPLY	multiplica o acumulador
40	BRANCH	desvio incondicional
41	BRANCHNEG	desvio se acumulador negativo
42	BRANCHZERO	desvio se acumulador zero
43	HALT	encerra execução

Exemplo SML (Exercício 7.27a – soma de 10 inteiros positivos com sentinela):

```

1 00 +1015  /* READ no local 15      */
2 01 +2015  /* LOAD valor lido      */
3 02 +4112  /* BRANCHNEG para 12 (sentinela negativa =
   fim) */
4 03 +2014  /* LOAD soma              */
5 04 +3015  /* ADD valor              */
6 05 +2114  /* STORE soma             */
7 06 +4000  /* BRANCH para 00 (proximo) */
8 12 +1114  /* WRITE soma              */
9 13 +4300  /* HALT                  */
10 14 +0000  /* soma                  */
11 15 +0000  /* valor lido             */

```

Estrutura do simulador (Ex. 7.28):

```

1  /* Ciclo de execucao da instrucao */
2  while ( !halt ) {
3      instructionRegister = memory[ instructionCounter
4          ];
5      operationCode = instructionRegister / 100;
6      operand      = instructionRegister % 100;
7
8      switch ( operationCode ) {
9          case READ:
10             scanf( "%d", &memory[ operand ] );
11             break;
12         case LOAD:
13             accumulator = memory[ operand ];
14             break;
15         case ADD:
16             accumulator += memory[ operand ];

```

```
16         break;
17     case BRANCH:
18         instructionCounter = operand;
19         continue; /* nao incrementa o IC */
20     case HALT:
21         halt = 1;
22         break;
23         /* ... demais casos ... */
24     }
25     ++instructionCounter;
26 }
```

Reflexão: Construir um simulador de máquina é uma das experiências mais formativas em ciência da computação. Conecta hardware (ciclos de busca/execução), arquitetura (organização de memória, registradores) e linguagens (interpretação de códigos de operação). Os Exercícios 7.27–7.29 são extensos – o livro dedica páginas a eles – mas o esforço vale muito a pena.

21. Exercício 7.30 – Cálculos Geométricos com Ponteiros para Função

Resposta detalhada

```
1  /* Ex7.30: geometria_funcoes.c
2     Menu com array de ponteiros para funcao */
3  #include <stdio.h>
4  #define PI 3.14159
5
6  void circunferencia( double raio );
7  void areaCirculo    ( double raio );
8  void volumeEsfera   ( double raio );
9
10 int main( void )
11 {
12     /* Array de ponteiros para funcao */
13     void ( *calculos[ 3 ] )( double ) = {
14         circunferencia, areaCirculo, volumeEsfera
15     };
16
17     int escolha;
18     double raio;
19
20     do {
21         printf( "\n0 - Circunferencia\n" );
22         printf( "1 - Area do circulo\n" );
23         printf( "2 - Volume da esfera\n" );
24         printf( "3 - Encerrar\n" );
25         printf( "Escolha: " );
26         scanf( "%d", &escolha );
27
28         if ( escolha >= 0 && escolha <= 2 ) {
29             printf( "Digite o raio: " );
30             scanf( "%lf", &raio );
31             ( *calculos[ escolha ] )( raio );
32         }
33     } while ( escolha != 3 );
34
35     return 0;
36 }
37
38 void circunferencia( double raio )
39 {
40     printf( "Circunferencia (r=%.2f) = %.4f\n", raio,
41         2 * PI * raio );
42 }
43
44 void areaCirculo( double raio )
45 {
```

```
45     printf( "Area (r=%.2f) = %.4f\n", raio, PI * raio
46         * raio );
47 }
48 void volumeEsfera( double raio )
49 {
50     printf( "Volume da esfera (r=%.2f) = %.4f\n",
51         raio, ( 4.0 / 3.0 ) * PI * raio * raio *
52         raio );
53 }
```

22. Exercício 7.31 – Cálculos com Dois Operandos

Resposta detalhada

```
1  /* Ex7.31: calculadora.c */
2  #include <stdio.h>
3
4  void somar      ( double a, double b );
5  void subtrair   ( double a, double b );
6  void multiplicar( double a, double b );
7  void dividir    ( double a, double b );
8
9  int main( void )
10 {
11     void ( *operacoes[ 4 ] )( double, double ) = {
12         somar, subtrair, multiplicar, dividir
13     };
14
15     int escolha;
16     double a, b;
17
18     do {
19         printf( "\n0-Somar 1-Subtrair 2-Multiplicar 3-
20             Dividir 4-Sair\n" );
21         printf( "Escolha: " );
22         scanf( "%d", &escolha );
23
24         if ( escolha >= 0 && escolha <= 3 ) {
25             printf( "Digite dois numeros: " );
26             scanf( "%lf%lf", &a, &b );
27             ( *operacoes[ escolha ] )( a, b );
28         }
29     } while ( escolha != 4 );
30
31     return 0;
32 }
```

```
33 void somar( double a, double b )
34 {
35     printf( "%.2f + %.2f = %.2f\n", a, b, a + b );
36 }
37 void subtrair( double a, double b )
38 {
39     printf( "%.2f - %.2f = %.2f\n", a, b, a - b );
40 }
41 void multiplicar( double a, double b )
42 {
43     printf( "%.2f * %.2f = %.2f\n", a, b, a * b );
44 }
45 void dividir( double a, double b )
46 {
47     if ( b == 0.0 ) {
48         printf( "Erro: divisao por zero!\n" );
49     }
50     else {
51         printf( "%.2f / %.2f = %.2f\n", a, b, a / b );
52     }
53 }
```